

第7组Vision文档

<GitHub 仓库问答与 Issue 分析系统>

前景文档

版本 <1.0>

修订历史记录

日期	版本	说明	作者
<09/07/2026	<1.0>	<完成初稿，按照标准前景文档模板完整填充项目高层需求、用户、商机、特性、约束、质量属性、优先级等全部章节>	<李明睿>

目录

- 1.简介4
 - 1.1目的4
 - 1.2范围4
 - 1.3定义、首字母缩写词和缩略语4
 - 1.4参考资料4
- 2.定位4
 - 2.1商机4

2.2问题说明4
2.3产品定位说明4
3.涉众和用户说明5
3.1涉众和用户概要5
3.2备选方案和竞争5
3.2.1<一个竞争对手>5
3.2.2<另一个竞争对手>5
4.产品概述5
5.产品特性5
5.1<一个特性>5
5.2<另一个特性>5
6.约束5
7.质量属性6
8.优先级6
9.其他产品需求（可选） 6
9.1适用的标准6
9.2系统需求6
9.3环境需求6
10.文档需求(可选)6
10.1用户手册6
10.2联机帮助6
10.3安装指南、配置文件、自述文件6

前景

1.1 目的

本前景文档用于收集、分析并定义 **GitHub 仓库问答与 Issue 分析系统** 的高层次业务需求、产品核心特性、用户价值、非功能质量约束与项目边界；统一开发小组、授课老师对系统目标、使用人群、业务范围的认知，作为界面原型开发、概要设计、编码实现、系统测试、课程验收的统一唯一依据，完整覆盖项目基础必达需求、进阶优化需求、创意拓展需求。

1.2 范围

本愿景文档所涵盖的范围为本系统的设计、开发、测试与交付。

1.2.1 关联项目与外部依赖

本系统并非孤立存在，其研发与运行高度依赖以下外部系统与组件：

GitHub 平台：本系统作为第三方辅助工具，通过 GitHub API、OAuth 认证和 Webhook 机制与目标开源代码仓库进行深度对接，实现数据双向同步。

大语言模型（LLM）及 RAG 引擎：本系统依赖外部大语言模型接口，通过检索增强生成（RAG）技术与向量数据库，提供代码语义理解与问答能力。

安全沙箱运行环境：本系统在执行代码自动修复时，需要调用隔离的沙箱环境（如 Docker 容器）进行依赖安装、代码编译和测试脚本运行，以确保修复方案的安全与可用性。

1.2.2 受本文档影响的对象与产物

本文档定义的系统边界、功能特性和技术约束，将直接影响并指导以下工作：

后续软件过程产物：本文档是后续编写系统架构设计方案、数据库设计、API 接口文档以及测试用例的直接输入与基准。

开发小组成员：明确了系统的核心功能与进阶功能边界，用于指导系统原型设计、模块划分、开发排期与技术选型，防止项目开发过程中出现需求蔓延。

指导老师/评审专家：作为项目阶段性检查、期末答辩以及最终软件系统验收的对照标准与核心评分依据。

1.3 定义、首字母缩写词和缩略语

为了便于正确理解和评审本愿景文档，以下对系统涉及的核心术语、首字母缩写词和缩略语进行了定义：

术语/缩写词	英文全称	解释 / 定义
LLM	Large Language Model	大语言模型。 基于海量文本与代码数据训练的深度学习模型，具备强大的自然语言语义理解、文本生成及代码编写能力。
Agent	-	智能体。在本系统中特指基于大语言模型驱动，具备目标感知、自主决策、记忆检索及外部工具调用能力的软件实体。
Multi-Agent	Multi-Agent System	多智能体系统。由多个不同职责的 Agent（如分类 Agent、问答 Agent、编码 Agent）通过分工合作来共同解决问题的系统。
RAG	Retrieval-Augmented Generation	检索增强生成。一种结合检索技术与大模型生成的架构。通过先从外部知识库检索出与问题相关的代码或文档，再将其作为上下文输入给模型，以减少模型“幻觉”并提高回答的准确性。
Issue Triage	-	

		Issue 分类与筛选。对代码托管平台上新提交的 Issue 进行内容分析，判定其类型（如 Bug 修复、功能请求、使用咨询、重复提交等）并为其打上对应标签的过程。
PR	Pull Request / Merge Request	拉取/合并请求。开发者修改代码后，请求项目维护者审查并将其分支代码合并到主分支（如 main/master）的一种协作机制。
Webhook	-	网络钩子。一种基于 HTTP 的事件驱动回调机制。当 GitHub 仓库发生特定事件（如新建 Issue、提交 PR）时，GitHub 会实时向本系统发送 HTTP POST 请求。
Sandbox	Sandbox Environment	沙箱环境。一种隔离的、受安全限制的虚拟运行环境（如 Docker 容器）。系统在此环境中运行待验证的测试脚本或编译生成的代码，以保障宿主机的安全。
Knowledge Graph	Knowledge Graph (KG)	知识图谱。一种用图结构存储和表达实体及其实体间关系的方法。本系统中用于结构化地表示代码仓库的模块划分、依赖关系、文件目录及测试关联。
FAQ	Frequently Asked Questions	常见问题解答。针对社区用户频繁提出的共性问题，提炼出的标准化问题与标准答案集合。

1.4 参考资料

- ☒ 《软件工程导论》Rational 前景文档编写规范、UML 用例建模标准；
- ☒ GitHub 官方开放 API 开发文档、OAuth 应用配置、Webhook 使用手册；
- ☒ FastAPI 官方开发文档、React+Ant Design 前端开发开发手册；
- ☒ ECharts 可视化图表官方示例、响应式页面开发 W3C 规范；
- ☒ 课程迭代计划、界面原型交付要求、Use-Case 用例规约文档；
- ☒ Git 版本管理标准化提交注释规范。

2. 定位

2.1 商机

2.1.1开源与企业内部共建生态的庞大基数

全球开源项目数量呈指数级增长，90% 以上的现代软件都依赖开源组件。同时，越来越多的大中型企业在内部推行“内部开源”模式，以复用组件、消除部门间技术壁垒。然而，由于缺乏专职维护人员，这些公开或内部的仓库积压了大量的技术债和未处理的 Issue。如何以极低的人力成本盘活这些资产，是当前整个行业面临的重要课题，这为本平台提供了广阔的用户基底。

2.1.2协同维护类 AI 智能体（Agent）的市场空白

目前的 AI 辅助开发工具（如 GitHub Copilot、Cursor 等）主要侧重于“IDE 端的个人编码辅助”。然而，面向项目团队、聚焦于仓库整体运维、Issue 治理、社区协同答复以及闭环自动修复（PR）的协作型 AI 智能体平台仍处于发展初期的探索阶段。本平台定位于该差异化生态节点，具备先进的先发优势与差异化竞争力。

2.1.3 软件维护者的刚性需求

GitHub 原生平台仅提供代码托管、基础 Issue 评论功能，无代码语义理解能力；开源维护者面对大量 Issue、代码提问时，需要手动翻阅海量源码、历史工单，维护成本极高；重复咨询类 Issue 反复出现，占用维护者大量重复沟通时间；Bug 类 Issue 定位故障代码、编写修复方案完全依赖人工，效率低下；多仓库并行维护时，无法统一管理所有项目的问答、工单记录。

2.2 问题说明

项目	详细内容
问题是	GitHub 原生工具缺少代码语义分析、智能工单处理、自动 Bug 修复能力，开源仓库维护者大量工作依赖人工重复操作
影响人群	开源 GitHub 项目作者、仓库维护人员、多仓库并行管理团队
问题的后果	1. 重复咨询类 Issue 反复人工回复，占用大量维护时间； 2. 查阅代码逻辑、定位故障 Bug 需要手动翻阅全部源码，效率低下； 3. 无自动工单分类、重复工单识别能力，工单管理杂乱； 4. 缺少仓库全局健康量化分析，无法直观掌握项目维护状态； 5. Bug 修复、分支创建、PR 提交全流程需要手动操作，流程繁琐
成功的解决方案核心优点	1. 一键绑定开源仓库，异步全量同步代码、工单、提交记录； 2. 自动构建向量知识库 + 代码知识图谱，支持语义化代码问答； 3. 实时监听新增 Issue，自动分类、生成标准化回复，一键推送至 GitHub； 4. Bug 工单自动定位故障代码，生成修复代码并自动创建 GitHub 修复分支与 PR； 5. 自动沉淀仓库长期记忆、聚类生成 FAQ，从源头减少重复咨询； 6. 可视化雷达看板量化仓库运维健康度，直观展示项目维护短板

2.3 产品定位说明

针对于	
-----	--

	开源软件维护者、开源社区治理团队，以及在内部推行开源共建且需维护大量共享组件的企业开发团队。
谁	饱受海量 Issue 积压、高频重复技术咨询的困扰，且在面对庞杂的代码仓库和历史上下文时，在定位 Bug、编写修复方案上需要耗费极高的时间与认知成本。
该（产品名）	基于多智能体协同的软件生命周期辅助与代码治理平台。
功能	能够自动解析代码结构与依赖关系以构建多维知识库，提供高精质的代码与文档智能问答；实时监听并自动分类、答复非代码 Issue；并能在安全沙箱中自动定位并尝试生成 Bug 修复代码与 Pull Request，从而有效减轻核心开发人员的日常维护压力。
不同于	传统的 IDE 端个人 AI 编码辅助插件（如 GitHub Copilot、Cursor）或单纯的静态代码扫描与工单流转系统。
我们的产品	提供了项目/仓库维度的全局上下文感知能力，采用多个分工明确的 Agent 开展协同（如客服 Agent、定位 Agent、编码 Agent），支持“从监听 Issue 到生成经过测试运行的修复 PR”的闭环自动化工作流，并具备学习团队特定编码规范和维护习惯的长期记忆能力。

3. 涉众和用户说明

3.1 涉众和用户概要

名称	说明	角色
系统开发小组	负责本助手平台的技术选型、架构设计、代码开发与测试的团队。	项目开发者
系统管理员（辅助用户）	后台全局管控人员，小组测试人员	查看全平台仓库同步日志、校验记忆数据、批量导出 FAQ 文档、排查同步失败故障，无代码问答、工单处理业务操作权限
普通社区用户	使用该开源软件，在 GitHub 上提问、反馈 Bug 或提交改进建议的最终用户（通过本系统的自动回复或问答功能与其产生交互）。	输入源与需求提出者

3.2 备选方案和竞争

维持现状（手工维护 + 传统规则机器人）：完全依赖人工筛选和人工回复，辅以传统的自动化脚本（。

使用集成化的云端 AI 协同环境：购买或接入托管平台原生提供的端到端智能工作流。

采用专用的自动化 Issue 修复 Agent：引入专注于“Issue 自动转化为 PR”的轻量化自动化机器人。

3.2.1 GitHub Copilot Workspace

主要优点：

生态集成度极高：作为 GitHub 官方的原生功能，用户无需额外安装复杂的第三方插件，可直接在网页端、GitHub 移动端或 CLI 中无缝调用。

交互式规划（Task-to-Plan-to-Code）：在生成代码前，会向用户展示一份可编辑的实施计划，支持人机协作（Human-in-the-Loop），降低了完全黑盒生成的失控风险。

模型能力强大：依托微软与 OpenAI/Anthropic 等头部厂商的合作，底层可灵活切换 GPT-4o、Claude 等顶尖模型。

主要缺点：

高昂的商业化成本：属于 GitHub Copilot 的高级增值功能，对于预算有限的中小型开源团队或个人维护者门槛较高。

定制化程度受限：难以根据单一特定项目的私有规范、历史偏好进行深度定制或长期记忆调优，缺乏针对特定项目群的 FAQ 库主动沉淀。

潜在的安全合规风险：官方 Agent 工作流在特定权限配置下，容易受到间接提示词注入（Prompt Injection）攻击，存在泄漏私有仓库信息的隐患。

3.2.2 Sweep AI

主要优点：

专注于缺陷自动修复：自动化闭环能力强，能够自动检索代码库、编写代码、在沙箱中运行 CI/CD 检查，并根据测试反馈进行自我纠错迭代，直至通过后才发起 PR。

轻量快捷：对小型团队和独立开发者极为友好，往往只需配置简单的 GitHub Action 标签（如打上 run-agent 标签）即可触发，即用即走。

主要缺点：

非代码 Issue 处理能力不足：主要侧重于代码级别的编写与修复（Issue-to-PR），对于大量的重复咨询、信息不全等“非代码日常 Issue”缺乏精细化的分类回复与过滤机制。

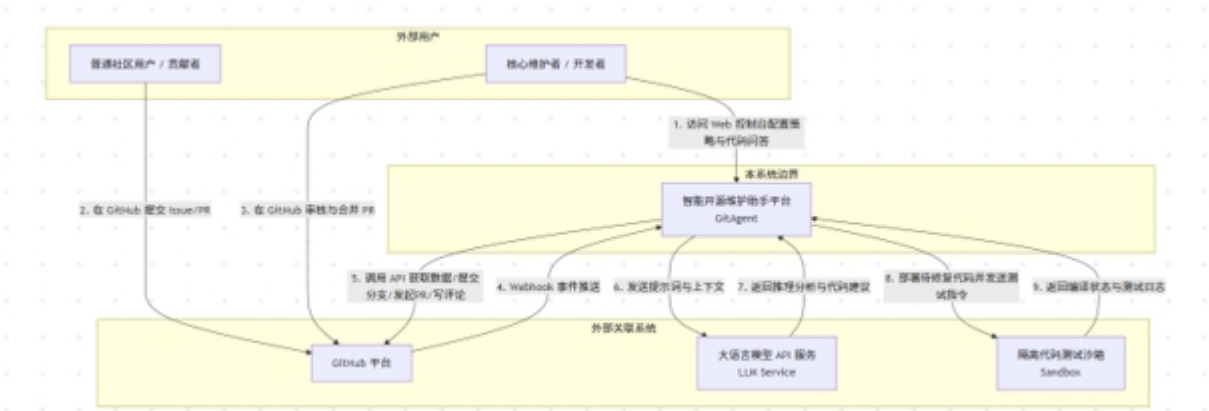
知识库交互维度单一：不支持多维度的“仓库智能问答（QA）”交互，无法帮助新贡献者通过日常对话形式快速理解整个代码仓库。

对团队规范适应慢：缺乏对团队历史行为和维护者审核偏好的长期记忆（Long-term Memory）模块，自动生成的代码风格有时需要人工进行二次重构。

4. 产品概述

本系统为前后端分离 Web 智能仓库运维平台，界面原型迭代阶段采用 HTML+CSS 编写静态响应式页面，正式开发使用 React+Ant Design 前端框架，后端基于 Python FastAPI 搭建；系统通过 OAuth 一键授权、Token 手动绑定两种方式对接 GitHub 开放 API，异步自动同步开源仓库源码、文档、Issue、PR、Commit 提交记录；系统通过 AST 语法树解析代码，构建向量知识库与代码知识图谱，结合大模型 RAG 多智能体实现精准代码问答；通过 GitHub Webhook 实时监听新增工单，自动完成 Issue 六分类、重复工单识别、标准化 AI 回复；针对 Bug 类工单自动定位故障代码、生成修复代码，自动创建独立修复分支、推送代码并关联原 Issue 创建 GitHub PR；系统持久化存储仓库长期记忆，定时聚类高频咨询自动生成 FAQ 知识库，配套 ECharts 雷达图实现仓库运维健康度可视化看板；系统区分仓库维护者、系统管理员两级权限，隔离全局日志管理功能。

系统外部交互对象：仓库维护者、系统管理员、GitHub 云端仓库、大模型 Embedding 与 LLM 云端接口；系统内部核心业务模块：仓库同步模块、知识库构建模块、智能问答模块、Issue 智能分析模块、Bug 自动修复模块、长期记忆 & FAQ 模块、可视化全局管理模块。



5. 产品特性

5.1 仓库数据无缝同步与感知

- **特性说明：**系统能够持续、自动同步所绑定 GitHub 仓库的底层资产，包括完整的代码库、项目文档、README、Issue、Pull Request 以及历史提交日志（Commit Log）。
- **为什么要实现该特性：**开源项目始终处于高速动态演进之中，任何人工手动上传最新代码或工单的操作都是繁琐且易错的。唯有保持多源数据资产的主动增量同步与准实时感知，才能保证智能体后续分析与决策的时效性和准确性。

项目代码多维智能问答

- **特性说明：**系统向用户提供交互式的代码对话窗口，能够准确解答有关项目架构“是什么（What）”、具体实现逻辑在“哪里（Where）”以及如何扩展或编写新功能“怎么做（How）”等维度的技术疑问。

- **为什么要实现该特性：**新晋贡献者或跨团队开发者在面对庞杂陌生的代码仓库时，理解代码的学习曲线极其陡峭，阅读源码耗时且耗力。智能问答能够在秒级提供具有代码引用的准确解答，显著降低技术人员上手项目的门槛，加速贡献周期的启动。

5.3 Issue 智能分流与非代码型自愈回复

- **特性说明：**系统自动、实时对新提交的 Issue 进行意图识别与分类（如缺陷汇报、新功能请求、配置疑问、信息不全、重复问题），并针对咨询类、环境缺失类或重复提交的 Issue 自动生成有事实依据的、礼貌规范的答复，提示用户或将其流转归档。
- **为什么要实现该特性：**日常维护中，非代码类的基础技术咨询与不规范的工单占用了维护者大量的宝贵精力。通过自动化分析与自愈回复，能在第一时间响应并拦截、过滤低价值工单，使用户得到即时反馈，同时将维护者从繁重机械的琐碎劳动中解放出来。

5.4 代码缺陷自动修复与拉取请求（PR）生成

- **特性说明：**针对需要修改代码的 Bug 或改进请求，系统能够结合历史记录定位潜在受影响的文件，在受控的临时运行环境中自动编写并应用修复方案；在通过既有测试验证后，自动在非主分支上创建代码提交并向仓库发起 Pull Request。
- **为什么要实现该特性：**快速修复软件缺陷是保障软件交付质量的核心环节。自动化的 Bug 定位、方案生成与沙箱跑测验证，不仅能够极大地压缩“发现 Bug 到修复 Bug”的等待周期，也免去了维护者人工下载、改写、本地测试常规基础漏洞的工作。

5.5 维护偏好学习与团队长期记忆

- **特性说明：**系统能够持续收集并记忆项目的历史 PR 合并偏好、既有 Linter/Format 规范以及维护者的习惯反馈，使得后续自动回复和自动修复代码的语气、风格自适应贴近原项目的既有标准。
- **为什么要实现该特性：**每个开源项目都有其独特的“社区文化”和“代码格调”。系统只有具备长期记忆与演进机制，才能表现得像一个经验丰富的、符合特定项目规范的“人类合作者”，避免自动生成的代码或回复因违背既有规范而增加人工审核与返工的摩擦成本。

6. 约束

为了保障系统的安全性、合规性与稳定性，在设计、开发与运行中必须遵循以下关键约束与外部依赖关系：

6.1 安全与执行隔离约束 (Security & Isolation Constraints)

- **沙箱执行约束：**自动修复代码和运行测试脚本的过程中，存在执行恶意代码（如破坏性命令、资源滥用等）的潜在风险。因此，所有涉及依赖安装、代码编译、测试脚本执行的自动化流程，必须在完全隔离且受限的容器化沙箱环境（如 Docker 容器、轻量级沙箱机制）中运行，以隔绝安全隐患。
- **权限最小化约束：**系统严禁拥有直接推送到目标仓库主分支（如 main/master）的直接写入权限。所有涉及代码级别的变更，系统必须且仅能通过创建独立分支并提交 Pull Request（PR）的方式提请修改，最终的合并权必须牢牢掌握在人类维护者手中。

6.2 法律、合规与开源许可证约束 (Legal & License Constraints)

- **许可证合规约束**：在自动生成代码或提供修复方案时，系统应确保不引入与目标项目开源许可证（License）相冲突的代码或第三方依赖。例如，避免在宽松的 MIT 协议项目中，自动合并引入了具有传染性的强 Copyleft（如 GPL 协议）的代码片段，防范潜在的法律纠纷。
- **敏感数据隐私保护（GDPR）**：系统在监听 Issue、PR 评论或拉取提交历史时，可能接触到用户无意泄露的敏感信息（如 API Key、私钥、个人敏感数据 PII）。系统应遵循主流的数据隐私保护规范，在后台处理中对潜在敏感数据进行脱敏或警告提示，不应将涉密数据持久化存储于模型训练或公共知识库中。

6.3 外部服务速率与性能限制约束 (External Service Rate Limits)

- **GitHub API 速率限制**：GitHub 官方对 API 调用有严格的频次限制（如经 OAuth 认证的接口通常为每小时 5000 次）。由于系统需要频繁拉取代码树、Issue、Commit 记录，系统设计必须引入本地缓存、增量更新与主动限流排队机制，防止因调用超限导致服务中断。
- **LLM API 吞吐与限流约束**：系统高度依赖大模型服务商的接口，大模型 API 普遍存在每分钟请求数（RPM）和每分钟 Token 数（TPM）限制，且存在网络延迟。架构设计必须支持异步非阻塞调用、合理的超时重试以及对推理结果的异常容错处理。

6.4 客户端与界面设计约束 (Client & UI Constraints)

- **Web 客户端约束**：系统客户端指定为标准的 Web 浏览器，不开发专用的原生桌面端或移动端 App。支持的主流浏览器应包括 Google Chrome、Mozilla Firefox、Apple Safari 和 Microsoft Edge。
- **多分辨率兼容约束**：前端界面必须采用响应式设计布局，确保在不同分辨率和屏幕尺寸的终端设备（包括高分辨率 PC 显示器、笔记本电脑、平板电脑及手机浏览器）上，功能菜单、代码对比视图、智能问答窗口等均能保持清晰、友好的布局和无障碍交互。

6.5 架构与并发处理约束 (Architecture & Concurrency Constraints)

- **高并发隔离约束**：由于需要同时并发处理多个不同的 GitHub 开源项目，系统的多 Agent 协同引擎、数据同步任务必须采用租户级别的队列隔离与进程（或协程）并发控制，防止单一仓库发生高频 Webhook 事件时占满全局系统资源，进而导致其他接入项目的服务产生排队或死锁。

7. 质量属性

本章定义了本项目在易用性、可靠性、性能、安全性、兼容性和可支持性等方面的非功能性需求，作为系统设计与测试验收的非功能指标。

7.1 易用性 (Usability)

- **一键集成与低配置成本**：系统应支持基于 GitHub OAuth 的快速接入，核心维护者应能在 3 分钟内完成仓库授权与基础同步策略配置，避免复杂的本地环境部署。

- **人机交互友好性**：Web 端的 RAG 问答界面需支持打字机流式（Streaming）渲染，并完整支持 Markdown、代码高亮渲染。
- **交互直观性**：系统在 GitHub 端发表的自动回复或 PR 说明，必须结构清晰，带有明确的“Bot 身份标识”并注明解答依据或引用源，避免引起社区用户的困惑。

7.2 可靠性 (Reliability)

- **高可用性与优雅降级**：系统整体可用性指标应达到 **99.5%** 以上。当依赖的外部大模型 API 出现暂时性故障或限流时，系统应能优雅降级（如对新 Issue 延迟队列重试，或仅标识标签而不发送空白/错误的回复），不影响 GitHub 本身的正常运转。
- **任务异常容错**：若在自动代码修复或沙箱测试运行过程中发生异常（如沙箱执行超时、依赖安装中断），系统应能自动销毁损坏的沙箱实例，记录错误日志，并向维护者发送诊断失败提示，避免提交不完整或无法编译的代码。
- **数据一致性**：Webhook 消息接收与处理队列需具备“至少交付一次（At-Least-Once）”保证，确保在系统短暂重启期间不会丢失 GitHub 推送的关键事件。

7.3 性能 (Performance)

- **高并发处理能力**：在系统推荐的服务器配置下，系统应能支持**同时并发接入 50 个以上**活跃的 GitHub 开源仓库，并平稳承载各仓库并发推送的 Webhook 请求，无明显的内存泄露。
- **响应时间 (Latency)**：
 - **Web 端 RAG 问答响应**：单次语义检索与大模型生成的首字响应时间控制在 **3 秒**以内，完整回答生成时间在 **15 秒**以内（取决于大模型服务商延迟）。
 - **Issue 智能分类与自动答复**：从 GitHub 监听到新 Issue 事件，到系统在 GitHub 对应页面自动提交 Comment 评论，整体处理延迟控制在 **3 分钟**以内。
 - **代码自动修复与 PR 发起**：从接收到缺陷 Issue 到系统完成本地定位、沙箱跑测并最终向 GitHub 提交修复 PR，整体耗时控制在 **15 分钟**以内（含沙箱内编译与运行测试脚本的时间）。

7.4 安全性 (Security)

- **凭据安全存储**：系统数据库存储的 GitHub 访问令牌（Access Token）、密钥（App Secrets）以及第三方大模型 API Keys，必须在持久化层采用强对称加密算法（如 AES-256）进行加密存储。
- **执行环境绝对隔离**：自动代码修复所调用的沙箱必须是瞬时（Ephemeral）的、无状态的容器。容器必须被限制 CPU 使用率、内存上限（例如限制单容器最大使用 2 核 CPU 和 4GB 内存）及网络带宽，并限制单次测试运行的最大执行时间（例如不超过 10 分钟），以防针对沙箱的恶意拒绝服务攻击（DoS）或容器逃逸。
- **提示词注入防御**：系统在将外部用户提交的 Issue 描述作为上下文输入给 Agent 前，应进行安全过滤和净化，防止利用恶意构造的 Issue 内容对大模型进行“提示词注入（Prompt Injection）”攻击，诱导 Agent 执行非授权操作。

7.5 兼容性 (Compatibility)

- **多浏览器兼容性：**Web 客户端必须无缝兼容各大主流现代浏览器，包括 Google Chrome、Apple Safari、Mozilla Firefox、Microsoft Edge，无排版错乱或 JS 运行错误。
- **响应式多分辨率适应：**前端界面必须能够完美适配多种常见显示分辨率，保证各种窗口和对话流自适应排版。
- **多语言技术栈支持：**虽然系统的语义理解受限于大模型本身的语料，但平台的基础图谱解析和 RAG 引擎应初步兼容至少三种主流语言技术栈（如 Python, Go, JavaScript/TypeScript）的项目结构和依赖规范。

8. 优先级

优先级	特性编号	特性名称	优先级归类原因
P0	FE-1	仓库数据无缝同步与感知	系统的基础数据通路。没有稳定的多源数据拉取与 Webhook 实时监听，后续的所有智能分析都将失去数据源。
P0	FE-3	项目代码多维智能问答	用户最直观的技术触达界面，解决新贡献者上手和维护者查阅代码的痛点，是系统生存的 MVP 核心功能。
P1	FE-4	Issue 智能分流与非代码型自愈回复	对应基础功能需求中的 Issue 监听与自动答复。能直接且大幅度减轻维护者处理低附加值工单的时间，核心效益高。
P2	FE-5	代码缺陷自动修复与 PR 生成	对应进阶需求。技术难度与风险高（涉及沙箱编译和测试执行），属于高价值、高难度的代码级自愈功能。
P3	FE-6	维护偏好学习与团队长期记忆	对应进阶需求。需要积累大量的历史 PR 和交互记录才能进行有效的模型微调或 Prompt 偏好关联，属于中后期的个性化体验调优。

9. 其他产品需求（可选）

9.1 适用的标准

9.1.1 网络通信与数据交互标准

- **安全传输协议 (HTTPS/TLS 1.2 & TLS 1.3)：**所有客户端与服务器、服务器与 GitHub API/大模型 API 之间的网络通信，必须强制采用 TLS 1.2 或更高版本的加密传输协议，确保数据在传输中不被窃听或篡改。

- **身份授权标准 (OAuth 2.0 - RFC 6749)**: 系统与 GitHub 平台之间的第三方登录及权限受控委托, 必须严格遵循 OAuth 2.0 框架协议, 保护用户账户凭证安全。
- **数据交换格式 (JSON - RFC 8259)**: 系统与 GitHub Webhook、前端 Web 控制台以及大模型 API 的接口数据交互, 统一采用标准 JSON 格式进行解析和序列化。
- **通信基础标准 (HTTP/1.1 & HTTP/2)**: Web 端接口服务和异步回调接收引擎需遵循标准 HTTP 传输协议规范。

9.1.2 Web前端与平台一致性标准

- **Web 核心标准 (HTML5 / CSS3 / ECMAScript 6+)**: 前端控制台代码编写必须遵循 W3C 组织发布的 HTML5、CSS3 及现代 JavaScript (ES6+) 标准规范, 以保证跨浏览器的一致性呈现与顺畅运行。
- **API 接口规范 (OpenAPI Specification 3.0)**: 平台内部各服务组件 (数据同步、知识库构建、多 Agent 协同等) 的 RESTful API 设计与文档定义, 遵循 OpenAPI 3.0 规范, 保障可扩展性。

9.1.3 安全与隐私保护合规标准

- **数据隐私保护法规 (GDPR / 《中华人民共和国个人信息保护法》)**: 由于系统会同步 GitHub 公开或私有项目中的 Issue、提交历史等数据 (可能包含开发者姓名、邮箱、头像等), 数据处理和持久化行为必须符合 GDPR 及国内相关法律关于个人信息收集、存储和去标识化的合规性要求。
- **Web 安全防护标准 (OWASP Top 10)**: 系统的后台接口和前端页面在开发时需遵循 OWASP (开放式Web应用程序安全项目) 定义的安全开发最佳实践, 防范 SQL 注入、跨站脚本 (XSS)、越权访问以及大模型特有的提示词注入 (Prompt Injection) 等高危安全漏洞。

9.1.4 软件工程与质量管理参考标准

- **语义化版本控制规范 (Semantic Versioning 2.0.0)**: 系统本身的版本迭代管理, 以及系统在为目标仓库自动发起 Bug 修复或合并变更时, 相关的 Git 标签、Commit Log 和 PR 说明应参考语义化版本规范, 维护项目历史日志的规整性。
- **软件质量模型标准 (ISO/IEC 25010)**: 作为系统非功能需求 (易用性、可靠性、性能效率、安全性、兼容性等质量属性) 定义与测试验收的框架参考标准。

9.2 系统需求

9.2.1 客户端需求

- **软件**: 支持 Chrome、Safari、Firefox、Edge 等主流浏览器。
- **设备**: 支持 PC、平板及手机, 界面需自适应不同屏幕尺寸。

9.2.2 服务端硬件需求

- **最低配置** (单项目测试): 4 核 CPU、8 GB 内存、50 GB SSD 存储。
- **推荐配置** (多项目并发): 8 核 CPU、16 GB 内存、200 GB SSD 存储。

9.2.3 服务端软件与运行环境

- **操作系统：**Linux (如 Ubuntu 22.04 LTS 或 Rocky Linux 9+)。
- **容器引擎：**Docker 20.10+（系统运行及临时测试沙箱的硬性依赖）。
- **技术栈与数据存储：**Python 3.10+、PostgreSQL 14+、Redis 6.2+、向量数据库（如 pgvector/ChromaDB）。

9.2.4 网络需求

- 需要公网出口以调用 GitHub API 和大模型 API，并能通过公网（HTTPS）正常接收 GitHub Webhook 推送。

9.3 环境需求

9.3.1 使用与用户环境

- **权限授权：**需在 GitHub 端完成 OAuth 授权，授予系统读取代码、读写 Issue 及创建分支/PR 的权限。
- **网络访问：**用户 Web 控制台交互和 GitHub 端 Bot 运行期间，网络不应对 WebSocket 和 HTTPS 传输进行封锁。

9.3.2 资源可用性

- **外部依赖：**GitHub 接口和第三方 LLM API 的网络连通性需保持稳定。
- **磁盘冗余：**服务端需常态化预留 20% 以上的 SSD 空间，用作代码拉取缓存与沙箱运行。

9.3.3 维护与日志管理

- **自动化清理：**系统需定时扫描并强制销毁超时的沙箱容器，日志需配置滚动清理，防止资源与磁盘泄露。
- **长期记忆备份：**对系统存储的项目开发偏好和 FAQ 知识库，需执行每日增量备份。

9.3.4 错误处理与恢复

- **沙箱失控保护：**若自动测试代码发生死循环，系统必须在安全时限（5-10分钟）内强制杀死该沙箱，释放资源。
- **故障容错：**服务器宕机重启后，系统必须能从 Redis 队列中自动恢复未完成的数据同步与分析任务。
- **抗抖动机制：**调用外部 API 失败时，需采用指数退避算法进行自动重试。